

SmartSwap: Measuring and Mitigating Background Interference on Unmodified Windows Desktops with a User-Mode Stall Index

Author One, Author Two, Author Three

Department, Institution

City, Country

email@institution.edu

Abstract—Background work on desktops (sync clients, indexers, updates, builds) degrades the tail latency of interactive applications. Windows gives unprivileged software no way to measure that harm or to tell which resource causes it. We present SmartSwap, a user-mode system that (i) estimates the share of wall time an interactive task loses to interference with a *Windows Stall Index* (WSI) measured by a low-duty canary, (ii) attributes the stall to CPU, memory bandwidth, or storage with a specificity-ordered differential rule, and (iii) applies a least-cost ladder of documented, reversible controls (scheduler hints, then an AIMD-governed Job Object CPU-rate cap). Every action is verified twice: by kernel read-back that it took effect, and by the canary that it helped. In a 340-trial randomised block study on a hybrid-core laptop, WSI tracked the tail latency of a separate, un-instrumented foreground with Spearman $\rho=0.96$. It detected interference with ROC AUC 0.965–1.000 and a 1.2 % false-alarm rate, and it named the injected resource in 86–99.6 % of stalled periods. The controller cut foreground p95 by 87 % under CPU and mixed contention, and was the only policy to help under storage contention (0.55× the p95 of a CPU-cap policy). Removing the ladder made p95 up to 4.8× worse. All 1 575 actions and 322 reverts were verified. Static priority demotion proved an equally effective baseline for latency, and we report where adaptive control over-throttled. We also dissect how our own first evaluation produced convincing but invalid results.

Index Terms—interference, tail latency, operating systems, Windows, feedback control, pressure stall information, performance evaluation.

I. INTRODUCTION

Interactive responsiveness is a tail-latency problem [1]. A user does not notice the average time to handle a keystroke or render a frame; they notice the slow ones. On a desktop, the slow ones are often caused by work the user did not start: a sync client hashing files, a search indexer, an update, a compiler. Operating systems expose resource *utilisation* (Task Manager on Windows), but utilisation is a poor proxy for harm. Memory-bandwidth contention, for example, can make an interactive task several times slower while half the cores sit idle. It is invisible to every utilisation counter Windows provides to user mode.

Linux addressed the measurement side with Pressure Stall Information (PSI) [2], which reports the share of time tasks are stalled on CPU, memory, or I/O. Datacentre systems such as Heracles, PARTIES, and Caladan [3, 4, 5] address the control side with feedback loops over hardware partitioning, assuming privileged control and instrumented services. A Windows

desktop has neither: no PSI equivalent, no cache or bandwidth partitioning exposed to applications, no administrator rights for ordinary software, and a foreground that cannot be instrumented.

This paper asks what can be done within those constraints. Our system, SmartSwap, contributes:

- 1) **A user-mode stall index for Windows (WSI).** A low-duty canary times its own scheduling delay, compute, memory streaming, and unbuffered storage reads. From these timings we derive the share of wall time an interactive task would lose, with no privileges, drivers, ETW sessions, or hardware counters. We validate it against a separate, un-instrumented foreground: $\rho=0.96$, $AUC \geq 0.965$ (§VII).
- 2) **Specificity-ordered differential attribution.** We show that a naive “largest slowdown wins” rule misattributes memory-bandwidth contention to storage. Ordering signals by specificity (CPU, then memory, then I/O) names the resource correctly in 86–99.6 % of stalled periods.
- 3) **A least-cost, doubly verified control ladder.** Scheduler hints come first; an AIMD-governed CPU-rate cap is used only if needed. Each action is checked by kernel read-back and by measured benefit, and is rolled back otherwise. A load-aware release rule prevents oscillation. An ablation shows the ladder is essential (up to 4.8× worse p95 without it).
- 4) **A cost-aware, randomised-block evaluation.** Every policy is measured on foreground tail latency *and* on background work lost. Comparisons are paired and Holm-adjusted, the control is genuinely free of interference, and validity checks confirm the injected load actually interferes. We include the strongest static baseline and report where it equals or beats our controller.
- 5) **A post-mortem of a flawed evaluation.** Our first evaluation reported significant improvements produced by six silent defects, including a workload that never ran. We describe each defect and the guard that now prevents it (§VIII).

§IV describes the design and §V the implementation. §VI gives the methodology and §VII the results. §IX discusses limitations.

II. RELATED WORK

Pressure signals. Linux PSI [2] exposes per-resource stall time from kernel accounting and underpins user-space

managers that throttle or kill work by cgroup. Windows has no user-mode equivalent: performance counters report utilisation and queue lengths, which are proxies for, not measurements of, lost foreground time. WSI recovers a PSI-like quantity from a probe's experience and adds attribution without privileges.

Interference control in datacentres. Heracles [3] and PARTIES [4] co-locate latency-critical and batch work under feedback controllers over cores, cache ways, memory bandwidth, and power. Caladan [5] reacts at microsecond scale using kernel-bypass signals. Dirigent [6] predicts progress to steer frequency and cache partitioning. Bubble-Up and Bubble-Flux [7, 8] use synthetic pressure probes to measure sensitivity. These systems assume server hardware partitioning (CAT, MBA), privileged control, and an instrumented latency-critical service. SmartSwap targets the opposite setting: a stock desktop, no privileges, no partitioning hardware, and a foreground that cannot be instrumented. It is limited to scheduler hints and CPU-rate caps, so it must decide *which* of these weak tools works and whether it actually helped.

Desktop controls. Windows exposes priority classes, I/O priority hints, Job Object CPU-rate limits [9], and EcoQoS [10]; scheduling internals are described in [11]. Commercial “process balancers” apply static or heuristic rules without verification or cost accounting. We include the strongest static rule (IDLE priority plus very-low I/O priority) as a baseline.

Measurement rigour. Systems measurements are easily biased by setup details [12]; rigorous practice calls for repetition, randomisation, and effect sizes with intervals [13,

14]. §VIII reports how our own first evaluation failed these standards.

III. PROBLEM AND THREAT MODEL

A desktop runs one latency-sensitive *foreground* (an editor, a browser tab, a game frame loop) next to a changing set of *background* work: builds, indexing, cloud sync, malware scans, updates. Background work is throughput-oriented; the user perceives only the foreground's tail latency [1]. We ask a narrow question: *using only documented, unprivileged user-mode controls on stock Windows, can a monitor detect when background work is hurting interactive tail latency, name the contended resource, and apply the cheapest control that removes the harm—while measuring what that control costs the background?*

We assume SmartSwap governs a known set of background processes (those it launched or was told to manage) and may not modify, inject into, or instrument the foreground. It may not use administrator rights, kernel drivers, ETW kernel sessions, or hardware performance counters. Every action must be reversible, and both the action and its reversal must be verified.

IV. DESIGN

Fig. 1 shows the pipeline. A *canary* feels contention on the user's behalf; the *Windows Stall Index* (WSI) and a *differential attribution* rule turn its timings into “how much” and “which resource”; a *controller* selects an action from a least-cost ladder; an *actuator* applies it to the governed group and verifies it by kernel read-back.

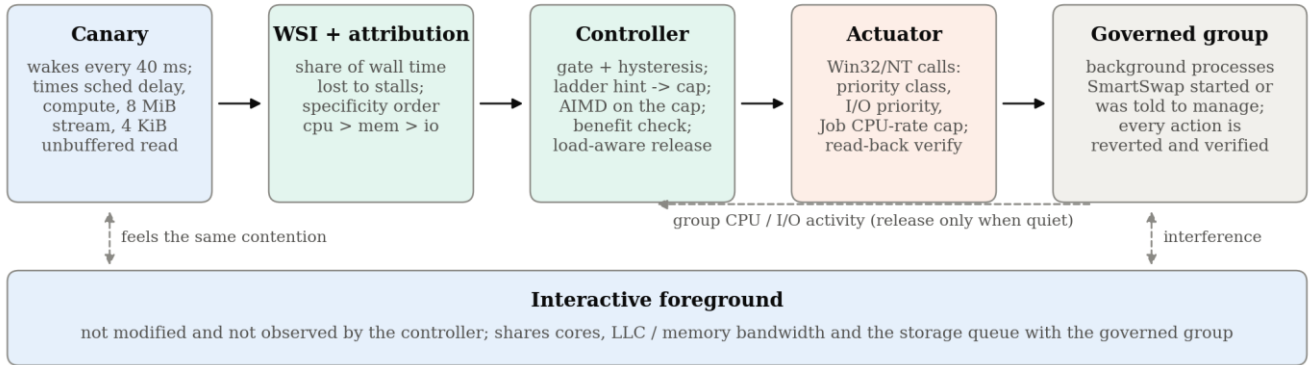


Fig. 1. SmartSwap v2. The controller never observes the foreground it protects; it reasons only from its own canary and from the activity of the processes it governs.

A. Canary and the Windows Stall Index

The canary is a separate low-duty process that wakes every $T=40$ ms and times four components of one “beat”: the scheduling delay s between its deadline and its actual wake-up, a fixed interpreter loop c (time-slice sensitive), a streaming copy m of an 8 MiB buffer (larger than the per-core L2, so last-level-cache and memory-bandwidth sensitive), and one 4 KiB unbuffered random read o opened with `FILE_FLAG_NO_BUFFERING` (storage-queue sensitive; the page cache cannot hide it). On an idle host SmartSwap records the median of each component, $\hat{s}, \hat{c}, \hat{m}, \hat{o}$, and their sum $\hat{\ell}$.

Over a sliding window W of the last eight beats, the WSI is the share of wall time the canary lost to interference:

$$\text{WSI} = 100 \cdot \frac{\sum_{i \in W} \max(0, s_i + c_i + m_i + o_i - \hat{\ell})}{|W|T} \% \quad (1)$$

WSI is a user-mode analogue of the Linux PSI “some” metric [2]: PSI counts time in which at least one task is stalled on a resource, using kernel instrumentation; WSI estimates the same quantity by direct experience of a probe task, which needs no privilege and works on any Windows build. The canary is in the spirit of Bubble-Up/Bubble-Flux probes [7, 8], but runs continuously on a desktop rather than profiling co-location offline.

B. Specificity-ordered differential attribution

Each component's *excess* is its window median relative to its idle median: $e_{cpu} = (\tilde{s} - \hat{s})/\hat{s}$, $e_{mem} = (\tilde{m} - \hat{m})/\hat{m}$, $e_{io} = (\tilde{o} - \hat{o})/\hat{o}$. A naive “largest excess wins” rule fails in practice. In our pilot runs, memory-bandwidth contention inflated the unbuffered read *more* than the memory stream itself (DMA and the I/O completion path compete for the memory controller), so it would have been misattributed to storage. The signals differ in *specificity*, not just magnitude: scheduling delay rises essentially only under CPU contention; the memory stream slows under bandwidth contention but barely under storage load; the read slows under both storage and bandwidth contention. SmartSwap therefore tests the most specific signal first and reports the first resource whose excess crosses a floor $\theta=1$ (the component is at least twice its idle median):

cause = first $r \in \{cpu, mem, io\}$ with $e_r \geq \theta$, else none (15)

C. Least-cost ladder with AIMD and benefit verification

Gating. The controller runs every 200 ms. It engages only after the WSI exceeds 5 % for two consecutive periods and a cause is named, which suppresses one-off spikes.

Level 1: resource-matched hint. The first action costs the background almost nothing when the foreground is mostly idle: for CPU and memory-bandwidth causes, the governed group's priority class is lowered to `IDLE_PRIORITY_CLASS`; for storage, its I/O priority hint is lowered to *very low* via `NtSetInformationProcess(ProcessIoPriority)`.

Mapping memory-bandwidth contention to a CPU-priority hint is an empirical choice. Bandwidth is not a scheduled resource, yet on our hybrid P/E-core pilot host this hint roughly halved probe p95 at negligible throughput cost. We report the effect and do not claim a mechanism (§IX).

Level 2: AIMD-governed CPU-rate cap. If the WSI stays above the SLO (3 %) for three consecutive periods under level 1, the controller places the group in a Job Object and sets a hard CPU-rate cap (`JOB_OBJECT_CPU_RATE_CONTROL_INFORMATION` [9]), starting at 50 % of machine cycles. The cap is then adapted by additive-increase/multiplicative-decrease [15]: $\times 0.6$ on an SLO violation (with a two-period hold-off so the window can refill) and $+5$ points per calm period. When it climbs back to 100 % it is removed and the ladder returns to level 1.

Benefit verification. SmartSwap verifies every action twice. The actuator re-reads kernel state after each call (priority class, I/O priority, job membership, cap flags and rate) and records whether it matches; this establishes that the action *took effect*. The controller then checks that it *helped*: if the mean WSI over the last five periods has not fallen by at least 30 % eight periods after a cap was set, the cap is rolled back as ineffective and not retried for ten seconds. A hard cap duty-cycles a job within each scheduling interval, so it can remove throughput without removing bursty interference; the benefit check stops such caps.

Load-aware release. A calm WSI under mitigation usually means the mitigation works, not that the load has left. Early prototypes that released after a calm hold relapsed

immediately and oscillated. SmartSwap knows which processes it governs, so it samples their CPU time and I/O bytes each period and releases hints only once the group is quiet (below 0.1 core and 1 MB/s). When group activity is unknown, it falls back to a calm hold that doubles after every relapse.

Safety. Actions target only the governed group and never raise priorities. Every action is reverted at disengagement or shutdown, and each revert is verified by read-back. Note that closing a Job Object handle does *not* lift a CPU-rate cap from processes still in the job, so the cap is explicitly cleared and the cleared state is read back.

V. IMPLEMENTATION

SmartSwap is a local FastAPI service with a static web dashboard (Python 3.14, ctypes bindings to kernel32 and ntdll; no third-party native code). The controller is a pure decision module with no Win32 calls, so it is unit-tested and can be replayed offline from recorded canary traces. Probe and worker programs avoid Python multiprocessing, for a reason that motivated part of this work (§VIII). Background workers stop together on a named manual-reset event, and they bucket completed work units by 100 ms of wall time so that throughput can be counted exactly inside the foreground's measurement window.

VI. EVALUATION METHODOLOGY

Foreground. A probe process emulates an interactive loop. Every 50 ms it wakes and performs a compute step (12 000 interpreter iterations), an 8 MiB memory stream, and a 4 KiB unbuffered random read from its own 64 MiB file. A beat's *response latency* is the sum of its scheduling delay and the three timed steps. A beat is a deadline miss above 25 ms. Each trial warms up for 1 s and then measures 6 s (120 beats). The primary metric is the trial's p95 response latency. The controller never sees this probe: its canary is a different process with a different period and work size.

Interference workloads. All workloads are bounded processes owned by SmartSwap: *CPU*: $2 \times$ the logical CPU count of spinning workers; *memory bandwidth*: one worker per two logical CPUs, each copying a 64 MiB buffer; *storage reads*: 16 workers issuing 1 MiB unbuffered random reads over a 1 GiB file (read-only, so no SSD wear); *mixed*: CPU, bandwidth, and storage workers together; *no interference*: a control with no background workers.

Policies. *None*: no mitigation. *Static priority*: every background worker is unconditionally set to `IDLE` priority class and very-low I/O priority. This is the strongest simple baseline, the Windows analogue of nice 19 with `ionice -c3`. *SmartSwap v1*: our earlier design, an unconditional 35 % Job CPU-rate cap plus `ABOVE_NORMAL` priority for the foreground. *SmartSwap v2*: the adaptive controller of §IV. *Observe-only*: v2's canary and controller run, but actions are suppressed. This measures monitoring overhead and gives attribution labels unconfounded by mitigation. *Ablations* (CPU, bandwidth, mixed): v2 without the ladder (straight to

the AIMD cap), without AIMD (fixed 50 % cap), and without benefit verification.

Cost. Each background worker counts completed work units, bucketed by wall time. We report *background work kept*: units completed inside the foreground's measurement window, relative to the no-mitigation trial of the same block. Each worker role is weighted equally in the mixed workload. A policy that protects the foreground by starving the background is visible as such.

Design and statistics. We use a randomised complete block design. Each block runs every (workload, policy) cell once, in a fresh order drawn from a seeded RNG, after re-calibrating the canary on an idle host. Blocking and randomisation spread slow drift (thermal state, turbo budget, unrelated desktop activity) across policies instead of letting it bias one of them [13, 14]. All comparisons are paired by block. For each we report the median of per-block p95 ratios with a 95 % percentile-bootstrap interval (10 000 resamples over blocks) [16] and an exact two-sided Wilcoxon signed-rank test on paired differences [17]. We apply Holm–Bonferroni adjustment [18] across *every* comparison reported in the paper as one family. Workloads are never pooled: each is its own experiment. Before each trial we sample non-study host CPU load for one second and keep it as a covariate.

Interference validity. A workload only supports conclusions about mitigation if it measurably hurts the foreground. We therefore first test unmitigated p95 under each workload against the no-interference control of the same block.

Host. One laptop: Intel Core Ultra 9 285H (hybrid, 16 cores and 16 threads, no SMT), 32 GB RAM, NVMe SSD, Windows 11 build 26200, AC power, *Balanced* power plan. We deliberately did *not* quiesce the desktop. OneDrive, Google Drive, and Windows Search were active throughout, as on a real user's machine. Controller parameters were set during pilot runs on this host and then frozen before the study; none were tuned on study data.

VII. RESULTS

The main study ran 340 trials (10 blocks $\times$ 34 cells) with no failed trials. Non-study host load before a trial had a median of 30 % CPU (p90 43 %). With $n=10$ blocks, the smallest exact two-sided Wilcoxon p is $2/2^{10}=0.002$, reached when all ten blocks agree in sign. Holm adjustment over the 37 comparisons of the main family therefore cannot go below 0.072. We report effect sizes with bootstrap intervals as the primary evidence, and give raw and adjusted p -values side by side; §IX discusses statistical power.

A. Do the workloads interfere?

Against the no-interference control of the same block, unmitigated foreground p95 rose $8.9\times$ under CPU contention ($3.95\rightarrow34.9$ ms, raw $p=0.004$) and $6.5\times$ under mixed interference ($p=0.002$). Memory-bandwidth contention raised p95 $2.6\times$ (10.8 ms), but one block's no-interference run was itself disturbed by unrelated desktop activity, so the test did not reach $p<0.05$ ($p=0.084$). Storage reads on this PCIe 4 NVMe drive inflated p95 by only $1.26\times$ ($p=0.23$). Storage results are therefore weak evidence, and we do not draw conclusions from them about mitigation.

B. WSI tracks the foreground's experience

The central claim of this paper is that a canary's stall index reflects interference felt by a *different* interactive process. Observe-only trials test this cleanly: the controller computes WSI but never acts. Across those 50 trials, trial-mean WSI and the foreground's p95 have Spearman $\rho=0.96$ ($p<0.001$; Fig. 2, left). Used as a per-period detector against no-interference periods, WSI has ROC AUC 0.965 (CPU), 0.997 (memory bandwidth), 0.995 (storage), and 1.000 (mixed) (Fig. 2, right). At the engage threshold of 5 % it fired in 1.2 % of no-interference periods. It fired in 95.1 %, 89.3 %, 82.9 %, and 100 % of interfered periods for the four workloads, respectively.

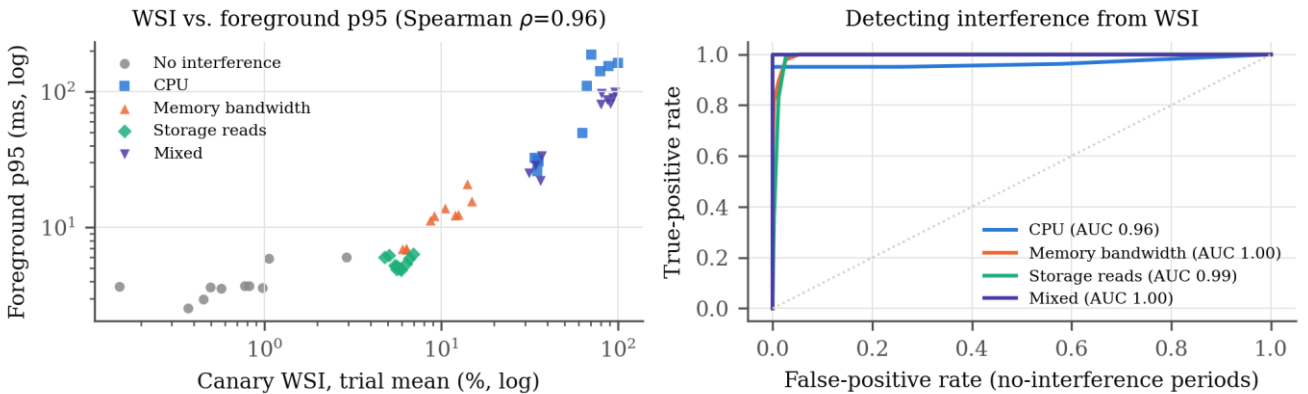


Fig. 2. Left: canary WSI against the separate foreground's p95, one point per observe-only trial (log–log). Right: ROC of WSI as an interference detector, per control period.

C. Attribution

In observe-only runs, among control periods whose WSI exceeded the engage threshold, the specificity-ordered rule

named the injected resource in 92.4 % (CPU), 85.8 % (memory bandwidth), and 99.6 % (storage) of periods (Table I). All 43 misattributed memory-bandwidth periods were labelled CPU, consistent with the scheduling delay that

bandwidth-bound workers also cause on a fully subscribed host.

TABLE I
ATTRIBUTION ACCURACY IN OBSERVE-ONLY TRIALS.

Workload	Periods $\geq$ engage threshold	Correctly attributed	Accuracy
CPU	330	305	92.4%
Memory bandwidth	302	259	85.8%
Storage reads	281	280	99.6%
No interference	periods above threshold: 1.2%		

D. Mitigation and its cost

Table II and Fig. 3 compare policies against no mitigation. Under CPU contention, all three mitigations cut median p95 by 87–91 %. SmartSwap v2 (ratio 0.13 [0.03, 0.16]) matched static priority (0.13) and kept 70 % of background work, against 51 % for v1's unconditional cap. Under mixed interference, all three reached 0.14 at 67 % background work. Under storage contention, v1's CPU cap had no effect (1.01),

because the workload is I/O-bound. v2 attributed the stall to storage and applied the I/O-priority hint, reducing p95 to 0.52 of the unmitigated value [0.47, 0.75] and to 0.55 of v1's [0.48, 0.71] (raw $p=0.002$, Holm 0.072). That protection cost 76 % of background read throughput: Windows' very-low I/O priority throttles aggressively.

Two results are less favourable to v2. First, **static priority is a strong baseline**. v2 is never significantly better than it on foreground latency (ratios 0.94–1.10, all CIs include 1). Second, under memory-bandwidth contention v2's level-1 priority hint did not bring WSI below the 3 % SLO, so v2 escalated to a CPU-rate cap. That cap passed the benefit check (WSI fell by at least 30 %) but did not improve the foreground beyond what priority alone achieved (0.60 vs. 0.57), and it cost 39 % of background work, where static priority kept 98 %. The canary was more sensitive to the remaining contention than the foreground was. A benefit check on WSI alone cannot detect this; we return to it in §IX.

TABLE II
FOREGROUND P95 BY WORKLOAD AND POLICY. RATIO: MEDIAN OVER BLOCKS OF POLICY P95 / NO-MITIGATION P95 IN THE SAME BLOCK, WITH 95 % BOOTSTRAP CI. p_{Holm} : EXACT WILCOXON, HOLM-ADJUSTED OVER ALL 37 COMPARISONS. BG KEPT: BACKGROUND WORK COMPLETED IN THE FOREGROUND WINDOW, RELATIVE TO NO MITIGATION.

Workload	Policy	p95 (ms)	p95 ratio vs. none [95% CI]	p_{Holm}	BG work kept
No interference	None	3.95	—	—	100%
	Static priority	3.86	0.89 [0.72, 1.00]	0.469	—
	SmartSwap v1 (static cap)	3.63	0.80 [0.56, 1.07]	1.000	—
	SmartSwap v2 (adaptive)	3.56	0.79 [0.56, 0.98]	0.742	—
CPU	None	34.92	—	—	100%
	Static priority	4.82	0.13 [0.03, 0.17]	0.072	66%
	SmartSwap v1 (static cap)	4.18	0.09 [0.02, 0.13]	0.072	51%
	SmartSwap v2 (adaptive)	4.66	0.13 [0.03, 0.16]	0.072	70%
Memory bandwidth	None	10.84	—	—	100%
	Static priority	6.76	0.57 [0.44, 0.97]	0.469	98%
	SmartSwap v1 (static cap)	6.40	0.57 [0.40, 0.79]	0.105	76%
	SmartSwap v2 (adaptive)	6.36	0.60 [0.43, 0.81]	0.146	61%
Storage reads	None	4.98	—	—	100%
	Static priority	2.91	0.58 [0.46, 0.81]	0.072	27%
	SmartSwap v1 (static cap)	5.07	1.01 [0.96, 1.06]	1.000	101%
	SmartSwap v2 (adaptive)	2.68	0.52 [0.47, 0.75]	0.105	24%
Mixed	None	29.90	—	—	100%
	Static priority	4.43	0.14 [0.08, 0.19]	0.072	67%
	SmartSwap v1 (static cap)	4.31	0.14 [0.08, 0.19]	0.072	67%
	SmartSwap v2 (adaptive)	5.02	0.14 [0.09, 0.21]	0.072	67%

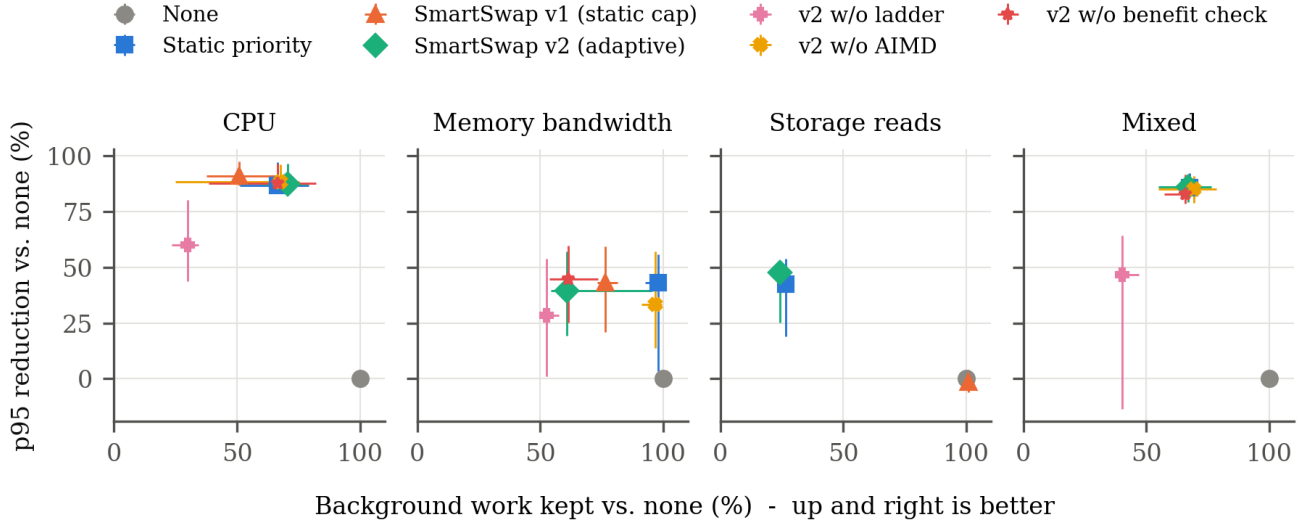


Fig. 3. Protection versus cost. Each marker is the median over ten blocks; whiskers are 95 % bootstrap intervals.

E. Ablations

Removing the ladder, so that the controller goes straight to an AIMD cap, was the only ablation with a large effect. It made p95 $4.8\times$ worse under CPU contention [3.4, 7.0] and $3.8\times$ worse under mixed interference [2.9, 7.9], while keeping only 30–40 % of background work (Table III). Hard caps duty-cycle the background, and its bursts still collide with the foreground. Removing AIMD (fixed 50 % cap) or the benefit check changed p95 by at most 11 %, with all intervals including or touching 1. In these stationary workloads the ladder rarely reached level 2 for long, so these two mechanisms had little to act on. The benefit check rolled back ten caps across the study.

TABLE III
ABLATIONS: P95 OF EACH VARIANT RELATIVE TO FULL v2 IN THE SAME BLOCK.

Workload	Variant	p95 ratio vs. full v2 [95% CI]	p_{Holt}	BG work kept
CPU	v2 w/o ladder	4.82 [3.38, 7.01]	0.072	30%
	v2 w/o AIMD	1.03 [0.95, 1.20]	1.000	67%
	v2 w/o benefit check	0.92 [0.83, 1.32]	1.000	67%
Memory bandwidth	v2 w/o ladder	1.07 [1.00, 1.37]	0.469	53%
	v2 w/o AIMD	1.11 [1.00, 1.18]	0.742	97%
	v2 w/o benefit check	0.98 [0.89, 1.02]	1.000	61%
Mixed	v2 w/o ladder	3.84 [2.94, 7.87]	0.072	40%
	v2 w/o AIMD	0.97 [0.86, 1.32]	1.000	69%
	v2 w/o benefit check	0.97 [0.90, 1.13]	1.000	66%

F. Overhead and verification

On an idle host, running the canary and controller *lowered* foreground p95 slightly (ratio 0.84 [0.67, 0.97]; p50 0.95). This is consistent with a periodic waker keeping cores out of deep idle states; we report it as an observation, not a benefit.

Every one of the 1 575 actions and 322 reverts in the study matched the kernel state read back after the call.

VIII. LESSONS FROM A FLAWED FIRST EVALUATION

SmartSwap v1 reported statistically reliable reductions of foreground p95 latency (e.g., 9.5 % under “CPU contention” and 16.2 % under “I/O contention”). An audit before this study found those results invalid, for reasons that apply well beyond this project:

- 1) *The load never ran.* CPU workers were started with Python multiprocessing from a `python -c` script. On Windows the `spawn` start method re-imports the target from `__main__`, which does not exist for `-c`, so every worker died at once. The background process exited after about 0.12 s, long before measurement began.
- 2) *The control was not a control.* The “no-pressure” preset used the same workload as “CPU contention”, so their indistinguishable p95 (7.2 vs. 8.3 ms) looked like a calm baseline.
- 3) *The validity gate measured the wrong thing.* Runs were labelled valid when probe p95 exceeded 5 ms. The probe’s own compute step took 4–5 ms, while scheduling delay stayed near 0.5 ms in every preset.
- 4) *Absence of action was hard-coded.* Zero actions in the control came from a preset-name check, not from detection.
- 5) *Reverts were asserted, not observed.* Closing a Job Object handle does not lift a CPU-rate cap from live members, yet the cap was logged as reverted.
- 6) *Statistics.* A fixed $z=1.96$ was used for paired intervals at every $n>5$, the baseline always ran before the treatment, and the cost to background work was never measured.

Each defect produced plausible-looking numbers; none raised an error. The v2 harness guards against all of them by construction. Workers handshake READY and report work done. The control has no workers. Interference is tested

against that control. Actions are decided only by the canary. Every action and revert is read back from the kernel. Order is randomised per block. Background throughput is measured in the same window as foreground latency.

IX. DISCUSSION AND THREATS TO VALIDITY

Measurement is the stronger contribution. The clearest result is that an unprivileged canary can stand in for an uninstrumented foreground ($\rho=0.96$, $AUC \geq 0.965$, 1.2 % false alarms), and that its components name the contended resource in 86–99.6 % of stalled periods. Windows exposes nothing like this to user mode today. A PSI-like signal is useful on its own: for diagnosis, for telemetry, and as input to any policy, including one that only advises the user.

Static priority is a strong baseline. On this host, lowering background priority recovers most of the attainable foreground latency for CPU, memory-bandwidth, and mixed contention, at modest cost. SmartSwap v2's advantages over it are qualitative rather than in p95. It acts only when WSI shows harm. It chooses the I/O hint only when storage is the cause. It verifies and reverts every change. It records evidence that the change helped. Practitioners should try priority demotion first; our ladder does exactly that.

A benefit check needs the right target. Under memory-bandwidth contention the cap lowered WSI by enough to pass the benefit check but did not improve the foreground further. Because the canary is more sensitive than the foreground to residual contention, a WSI-only SLO can over-throttle. Two remedies follow from the data. One is a cost-aware benefit test that keeps a cap only if the WSI reduction per unit of background throughput lost exceeds a bound; SmartSwap already measures the group's CPU time, so this is computable. The other is a resource-specific SLO.

Statistical power. Ten blocks give large, consistent effects with bootstrap intervals that exclude no effect, but a 37-test Holm family cannot reach $p < 0.05$ with $n=10$. A confirmatory study should pre-register a small primary family or use about 20 blocks.

External validity. All results come from one hybrid laptop on AC power in the *Balanced* plan, with a busy but uncontrolled desktop. The foreground is a synthetic probe rather than a real application, and the background workloads are synthetic. Storage interference was weak on a fast NVMe drive and may be stronger on SATA SSDs or hard disks. Controller parameters were chosen in pilot runs on the same host; they were frozen before the study, but they were not validated on unseen hosts. Replication on homogeneous-core and AMD hosts, real foreground applications (input-to-display latency), and real background agents (sync, indexing, builds) are the most important next steps.

Scope of control. SmartSwap governs only processes it starts or is explicitly given. Choosing *which* third-party processes to govern raises policy questions (consent, fairness, security software) that we leave open.

X. CONCLUSION

SmartSwap shows that stock Windows can measure the time interactive work loses to background interference from user mode, name the contended resource, and apply verified, reversible controls, with no privileges. On a hybrid laptop, the Windows Stall Index tracked an un-instrumented foreground's tail latency closely, and attribution was accurate. A least-cost ladder matched the strongest static baseline while acting only on evidence; removing the ladder made tail latency up to $4.8\times$ worse. We also documented how a first evaluation produced convincing but invalid results, and how the second one guards against each failure.

ARTIFACT

The service, controller, study harness, raw per-beat traces of every trial, and the analysis that regenerates every number, table, and figure in this paper are available at <https://github.com/ANONYMIZED> (anonymised for review).

REFERENCES

- [1] J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.
- [2] J. Weiner, "PSI – Pressure Stall Information," Linux kernel documentation, <https://docs.kernel.org/accounting/psi.html>, 2018.
- [3] D. Lo, L. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis, "Heracles: Improving Resource Efficiency at Scale," in *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, 2015.
- [4] S. Chen, C. Delimitrou, and J. F. Martinez, "PARTIES: QoS-Aware Resource Partitioning for Multiple Interactive Services," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [5] J. Fried, Z. Ruan, A. Ousterhout, and A. Belay, "Caladan: Mitigating Interference at Microsecond Timescales," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.
- [6] H. Zhu and M. Erez, "Dirigent: Enforcing QoS for Latency-Critical Tasks on Shared Multicore Systems," in *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2016.
- [7] J. Mars, L. Tang, R. Hundt, K. Skadron, and M. L. Soffa, "Bubble-Up: Increasing Utilization in Modern Warehouse Scale Computers via Sensible Co-locations," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2011.
- [8] H. Yang, A. Breslow, J. Mars, and L. Tang, "Bubble-Flux: Precise Online QoS Management for Increased Utilization in Warehouse Scale Computers," in *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*, 2013.
- [9] Microsoft, "Job Objects and JOBOBJECT_CPU_RATE_CONTROL_INFORMATION," Win32 API documentation, <https://learn.microsoft.com/windows/win32/procthread/job-objects>. Accessed 2026-10-06.
- [10] Microsoft, "Quality of Service (EcoQoS) and Process Power Throttling," Win32 API documentation, <https://learn.microsoft.com/windows/win32/procthread/quality-of-service>. Accessed 2026-10-06.
- [11] P. Yosifovich, A. Ionescu, M. E. Russinovich, and D. A. Solomon, *Windows Internals, Part 1*, 7th ed., Microsoft Press, 2017.
- [12] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing Wrong Data Without Doing Anything Obviously Wrong!," in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009.

- [13] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically Rigorous Java Performance Evaluation," in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 2007.
- [14] T. Kalibera and R. Jones, "Rigorous Benchmarking in Reasonable Time," in *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, 2013.
- [15] D. Chiu and R. Jain, "Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks," *Computer Networks and ISDN Systems*, vol. 17, no. 1, pp. 1–14, 1989.
- [16] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- [17] F. Wilcoxon, "Individual Comparisons by Ranking Methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [18] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.